Improved PINNs for option pricing: an implementation of Bai, Chaolu & Bilige (2022)

Sondre Rogde, CEE292 Project 1, November 2026

1 Introduction

Bai, Chaolu and Bilige [1] apply physics-informed neural networks (PINNs, [2]) to two option-pricing PDEs, the nonlinear Ivancevic equation and the Black–Scholes equation, and propose an "improved PINN" (IPINN) that adds a neuron-wise locally adaptive activation functions with slope recovery proposed in Jagtap et al. [3]. The authors claim that their IPINN model converges faster and is slightly more accurate than the plain PINN on a financial rogue wave, a soliton, and a European call.

This project implements both the PINN and the IPINN in PyTorch and runs the same three experiments as the paper with 5 random seeds each. We reproduce the paper's PINN results, but we are not able to reproduce the improvement it reports for the IPINN. Using the loss as written in the paper, the slope-recovery term takes over training and the IPINN diverges. Excluding the extra term fixes this, but are still not better than the plain PINN on these problems. Code and results: `https://github.com/REPO-LINK`.

2 Methods

2.1 PINN and IPINN

A fully connected network $\hat{u}_\theta(S,t)$ with 8 hidden tanh layers approximates the PDE solution. It is trained on the usual PINN loss we have seen in class:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{data}} + \mathcal{L}_{\text{physics}}, \quad \mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_i \left|\hat{u}_\theta(S_i,t_i) - u_i\right|^2, \quad \mathcal{L}_{\text{physics}} = \frac{1}{N_{\text{colloc}}} \sum_j \left|\mathcal{R}_\theta(S_j,t_j)\right|^2, \tag{1}$$

where the data points lie on the initial and boundary lines, the collocation points are random points spread evenly over the domain, and the residual $\mathcal{R}_\theta$ is the PDE evaluated on $\hat{u}_\theta$ with automatic differentiation. Training is Adam followed by L-BFGS.

The IPINN keeps this loss but replaces every hidden activation by $\tanh(n\,a\,z)$ with one trainable slope a per neuron ($n = 10$, $a = 0.1$ at initialization, i.e. a plain tanh network at the start) and adds the slope-recovery term of [3],

$$\mathcal{L}_{\text{IPINN}} = \mathcal{L}_{\text{total}} + S(a), \qquad S(a) = \left[\text{mean}_{\text{layers}}\, \exp\left(\text{mean}_{\text{neurons}}\, a\right)\right]^{-1}, \tag{2}$$

which is meant to push the slopes up and speed up training. Because this term turned out to break training (Section 3), we also run a third variant that the paper does not include where we remove the adaptive slopes term $S(a)$. It differs from the PINN only by the trainable slopes.

2.2 Problems

The Ivancevic model is the nonlinear Schrödinger equation $i\psi_t + \frac{\sigma}{2}\psi_{SS} + \beta|\psi|^2\psi = 0$ for a complex option-price wave function $\psi(S,t)$. The network outputs the real and imaginary parts and the residual has two components. As in the paper we solve for the financial rogue wave of Yan [4] ($\sigma = \beta = 1$ on $S \in [-2,2]$, $t \in [-1.5,1.5]$) and a bright soliton ($\sigma = \beta = 2$, $\psi = 2\,\text{sech}(2S+8t)\,e^{i(1-2S)}$ on $S \in [-5,5]$, $t \in [-0.5,0.5]$). The Black–Scholes equation is given by, $V_\tau = \frac{1}{2}\sigma^2 S^2 V_{SS} + rSV_S - rV$, with initial condition $V(S,0) = \max(S-K,0)$. We have $K = 10$, $r = 0.1$, $\sigma = 0.05$ on $S \in [0,20]$, $\tau \in [0,1]$. All three have closed-form solutions, which are used for the training data and the test errors.

2.3 Training and evaluation

Network sizes (8 hidden layers of 40 units for Ivancevic, 50 for Black–Scholes), the number of data points ($N_{\text{data}} = 100$ and 200) and the collocation points of the Ivancevic problems follow the paper. Table 1 lists where we deviate. The reductions were made so that all 45 runs complete in reasonable time on a laptop (Apple M2). I also did a check with 5000 Adam steps on one seed which did not change the results significantly. Errors are the relative L^2 error and the maximum absolute error on a dense grid, reported as mean ± std over the 5 seeds. The seed changes both the sampled points and the initialization.

Table 1: Our setup versus the paper's.

	this project	paper
Adam steps	2000 (lr 10^{-3})	10 000
L-BFGS iterations	up to 5000 (strong Wolfe, stops on tolerance)	15 000
N_{colloc}, Black–Scholes call	2000	10 000
Black–Scholes training data	closed-form solution	finite-difference solver
Precision	single	not stated
Runs per experiment	5 seeds, mean ± std	1

3 Results

Table 2: Test errors, mean ± std over 5 seeds.

problem	method	relative L^2 error	max. abs. error
Black-Scholes call	PINN	$(4.69 \pm 1.42) \times 10^{-4}$	$(6.19 \pm 1.32) \times 10^{-2}$
	IPINN w/o slope recovery	$(4.26 \pm 0.51) \times 10^{-4}$	$(6.21 \pm 0.67) \times 10^{-2}$
	IPINN	$(5.19 \pm 1.49) \times 10^{-3}$	$(4.98 \pm 4.41) \times 10^{-1}$
Ivancevic soliton	PINN	$(4.30 \pm 0.55) \times 10^{-2}$	$(1.42 \pm 0.20) \times 10^{-1}$
	IPINN w/o slope recovery	$(6.87 \pm 3.14) \times 10^{-2}$	$(2.36 \pm 1.05) \times 10^{-1}$
	IPINN	$(1.08 \pm 0.03) \times 10^{0}$	$(2.60 \pm 0.05) \times 10^{0}$
Ivancevic rogue wave	PINN	$(2.72 \pm 1.28) \times 10^{-2}$	$(9.40 \pm 3.99) \times 10^{-2}$
	IPINN w/o slope recovery	$(4.42 \pm 1.01) \times 10^{-2}$	$(1.49 \pm 0.33) \times 10^{-1}$
	IPINN	$(9.23 \pm 0.14) \times 10^{-1}$	$(3.74 \pm 0.01) \times 10^{0}$

Our results agree with the paper on the plain PINN and disagree on the IPINN. For the standard PINN, our rogue-wave error is below the 7.3×10^{-2} the original paper reports. The Black–Scholes error is an order of magnitude below their 8.2×10^{-3}. This is most likely because our training data is exact, while they used a finite difference solution. The soliton error is an order of magnitude above their 2.6×10^{-3}, which might be due to the smaller training budget.

The IPINN as written, however, diverges on both Ivancevic problems and is ten times worse than the PINN on Black–Scholes, on every seed. The cause is the slope-recovery term itself. $S(a)$ in Eq. (2) decreases toward 0 as the slopes grow, so the optimizer can always lower it by making the slopes larger, and nothing in the loss stops this. After the Adam phase $S(a)$ is still about 0.8 while the data and physics losses are between 10^{-4} and 10^{-2}, so $S(a)$ dominates the total loss and L-BFGS tries to fix this. It does this by increasing the a making the slopes steeper. This makes the network become nearly constant as it is stuck at -1 or 1 almost everywhere. Figure 1 shows this. The data-and-physics part of the loss (plotted without $S(a)$) follows the PINN during Adam but rises as soon as L-BFGS starts. Without $S(a)$ the slopes stay near 1 and the adaptive network trains like the plain PINN.

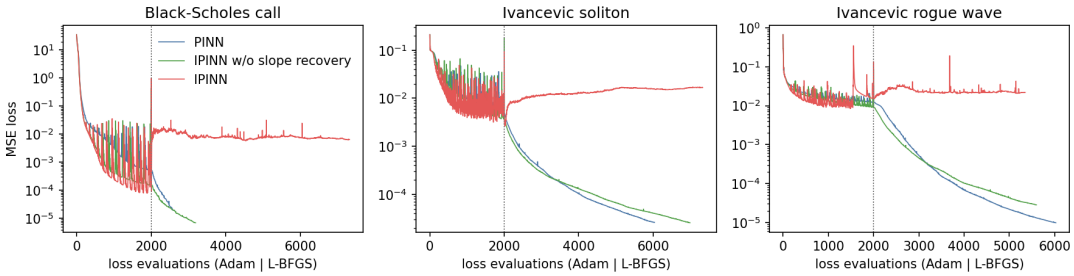


Figure 1: Data-and-physics part of the loss, $\mathcal{L}_{\text{total}}$ without $S(a)$, during training for seed 0. The dotted line marks the switch from Adam to L-BFGS. For the paper's IPINN this part rises during L-BFGS because the optimizer lowers $S(a)$ by inflating the slopes instead.

4 Conclusion

The plain PINN from the paper was straightforward to reproduce and works on all three problems with a much smaller training budget than the paper used. The improved version did not work for us. It seems like the extra loss term rewards ever larger activation slopes. The paper does not report the learned slopes or any bound on the term, so it is hard to tell how its runs avoided this. It might be avoided by weighting the slope-recovery term differently. More plots and results can be found in the GitHub repo.

References

[1] Y. Bai, T. Chaolu, S. Bilige. The application of improved physics-informed neural network (IPINN) method in finance. *Nonlinear Dyn.* 107, 3655–3667 (2022).

[2] M. Raissi, P. Perdikaris, G. E. Karniadakis. Physics-informed neural networks. *J. Comput. Phys.* 378, 686–707 (2019).

[3] A. D. Jagtap, K. Kawaguchi, G. E. Karniadakis. Locally adaptive activation functions with slope recovery for deep and physics-informed neural networks. *Proc. R. Soc. A* 476, 20200334 (2020).

[4] Z. Yan. Financial rogue waves. *Commun. Theor. Phys.* 54, 947–949 (2010).